Java Concurrency Portfolio Project

Brendon Williams

Colorado State University Global

CSC450: Programming III

Instructor Hasteltine

8/2/2026

Java Concurrency Portfolio Project

Program Overview

This project demonstrates the use of concurrency in Java by creating two separate threads that act as counters. The first thread counts upward from 0 to 20. After the first thread completes its execution, a second thread begins counting downward from 20 to 0. The program uses Java's built-in *Thread* class to create and execute each thread while using the *join()* method to synchronize their execution.

The purpose of this application is to demonstrate the basic principles of multithreaded programming. By requiring the second thread to wait until the first thread has finished, the program illustrates how synchronization controls execution order and prevents both threads from writing to the console simultaneously. This approach produces predictable output while introducing important concurrency concepts used in larger Java applications.

Pseudocode

BEGIN PROGRAM

1. Import the required Java thread classes.
2. Create a method named countUp().
   a. Display "Thread 1: Counting Up."
   b. Count from 0 to 20.
   c. Display each number.
   d. Pause briefly between each count.

3. Create a method named countDown().

   a. Display "Thread 2: Counting Down."

   b. Count from 20 down to 0.

   c. Display each number.

   d. Pause briefly between each count.

4. Begin the main method.

   a. Display the program title.

   b. Create the first thread using countUp().

   c. Start the first thread.

   d. Wait for the first thread to finish using join().

   e. Create the second thread using countDown().

   f. Start the second thread.

   g. Wait for the second thread to finish using join().

 h. Display "Program completed successfully."

END PROGRAM

Java Source Code

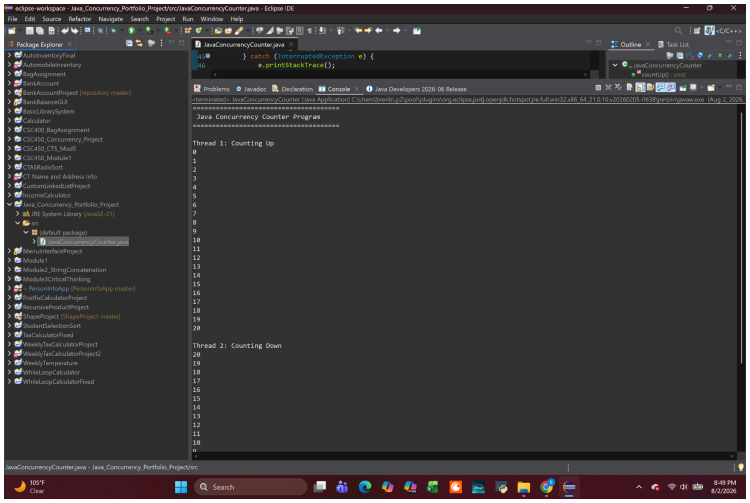
```
public class JavaConcurrencyCounter {


    // Thread 1 counts from 0 to 20.
    public static void countUp() {
        System.out.println("\nThread 1: Counting Up");


        for (int i = 0; i <= 20; i++) {
            System.out.println(i);
```

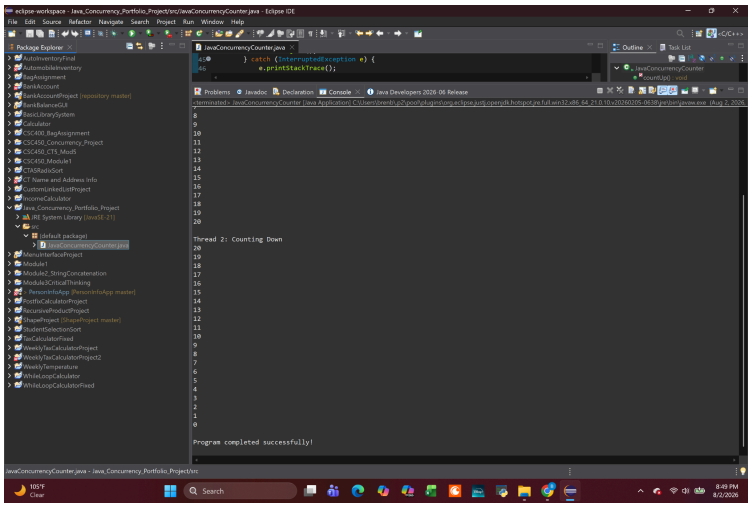
```
        try {
            Thread.sleep(200);
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted.");
        }
    }
}

// Thread 2 counts from 20 to 0.
public static void countDown() {
    System.out.println("\nThread 2: Counting Down");

    for (int i = 20; i >= 0; i--) {
        System.out.println(i);

        try {
            Thread.sleep(200);
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted.");
        }
    }
}

public static void main(String[] args) {
```

```
System.out.println("=======================================");
System.out.println(" Java Concurrency Counter Program");
System.out.println("=======================================");

Thread thread1 = new Thread(() -> countUp());

thread1.start();

try {
    thread1.join();
} catch (InterruptedException e) {
    e.printStackTrace();
}

Thread thread2 = new Thread(() -> countDown());

thread2.start();

try {
    thread2.join();
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

```
        System.out.println("\nProgram completed successfully!");
    }
}
```

Program Execution





Program Execution Results

The program executed successfully and produced the expected output. The first thread began by counting upward from 0 to 20. After reaching 20, the first thread completed and the second thread began counting downward from 20 to 0. Once both threads finished, the program displayed a completion message and terminated normally.

The output confirms that the *join()* method correctly controlled the execution order. The second thread did not begin until the first thread had completely finished. This prevented the counter values from appearing in a mixed or unpredictable order and demonstrated proper thread synchronization.

Performance Issues with Concurrency

Concurrency can improve performance when an application contains tasks that can run independently at the same time. On a multi-core processor, separate threads may execute on different cores, which can reduce the total time needed to complete certain workloads. However, concurrency also introduces overhead. The Java Virtual Machine and operating system must create, schedule, pause, and resume threads. This process requires memory and processor time.

Context switching is another potential performance issue. A context switch occurs when the processor stops executing one thread and begins executing another. If an application creates too many threads, frequent context switching may reduce performance instead of improving it.

Threads may also compete for shared resources, causing delays when synchronization mechanisms such as locks are required.

In this program, the performance cost is small because only two threads are created. The threads do not run in parallel because the second thread waits for the first thread to finish. Therefore, this application does not gain a speed advantage from concurrency. Its purpose is to demonstrate thread creation and synchronization. The short delay created by *Thread.sleep()* also intentionally slows the program so that the output can be observed. In a production application, unnecessary sleep calls should be avoided because they increase execution time.

Vulnerabilities Exhibited with Use of Strings

Although this application does not process user input, string handling remains an important security consideration in Java. Java uses the *String* class, which is immutable. Once a *String* object has been created, its contents cannot be modified. This immutability helps prevent many memory corruption vulnerabilities that are common in lower-level programming languages.

However, immutable strings also have disadvantages when storing sensitive information. Passwords, encryption keys, and other confidential data stored in *String* objects may remain in memory until they are removed by Java's garbage collector. Because developers cannot directly overwrite a *String*, sensitive information may remain accessible in memory longer than intended.

For applications that process confidential information, Java developers often use *char[]* instead of *String*. Character arrays allow the data to be erased immediately after use, reducing the amount of time sensitive information remains in memory. Understanding how strings are stored and managed helps developers build more secure Java applications.

Security of the Data Types Exhibited

This application primarily uses integer variables and Java *Thread* objects. The integer data type is appropriate because the counter values remain within a very small range, eliminating any realistic possibility of integer overflow. Strongly typed variables improve code readability and reduce programming errors caused by unintended type conversions.

The *Thread* class provides a reliable framework for creating concurrent applications. Proper synchronization through the *join()* method ensures that each thread completes execution before the next begins. The application also catches *InterruptedException*, preventing unexpected thread interruptions from terminating the program without proper handling.

Selecting appropriate data types contributes to both application reliability and security. Strong typing, proper exception handling, and thread synchronization reduce the likelihood of runtime errors and improve overall software quality.

Comparison Between the Java and C++ Implementations

The Java and C++ versions of this project accomplish the same objective by demonstrating concurrency through the use of two synchronized threads. In both applications, the first thread counts upward from 0 to 20 before the second thread begins counting downward from 20 to 0. Although the final output produced by both programs is identical, the two programming languages differ considerably in their implementation, performance, memory management, security, and ease of development. Understanding these differences helps developers choose the most appropriate language based on the requirements of a project.

One of the primary similarities between the two implementations is the use of separate threads to perform independent tasks. In the Java application, concurrency is implemented using the *Thread* class, while synchronization is achieved with the *join()* method. The Java Virtual Machine manages thread creation and scheduling, allowing developers to focus primarily on program logic rather than low-level operating system interactions. This makes Java relatively straightforward for developers who are learning concurrency concepts or building enterprise applications where portability and reliability are important.

The C++ implementation uses the Standard Library's *std::thread* class to create concurrent threads. Similar to Java, the *join()* function is used to ensure that the second thread does not begin execution until the first thread has completed. Unlike Java, however, C++ executes directly as native machine code without requiring a virtual machine. Because of this design, C++ gives programmers greater control over thread behavior and system resources. While this additional control can improve performance, it also increases the complexity of

development and places more responsibility on the programmer to correctly manage threads and shared resources.

From a performance standpoint, C++ generally has an advantage over Java. Since C++ applications are compiled directly into machine code before execution, they typically execute faster and require less memory than equivalent Java applications. There is no virtual machine or garbage collector running in the background, reducing the amount of overhead introduced during execution. These characteristics make C++ a popular choice for operating systems, game engines, embedded systems, and other applications where maximum performance is required.

Java applications execute within the Java Virtual Machine, introducing an additional software layer between the application and the operating system. Although this creates some overhead, the Java Virtual Machine performs many optimizations, including Just-In-Time compilation, which significantly improves runtime performance. For applications such as business software, web services, and enterprise systems, the performance difference between Java and C++ is often negligible because modern hardware and JVM optimizations minimize much of the additional overhead. In this portfolio project, the difference in execution speed between the two programs would likely be unnoticeable because both applications perform only a small amount of work.

Memory management is another area where the two languages differ significantly. Java automatically manages memory through garbage collection. When objects are no longer being used, the garbage collector eventually frees the memory without requiring direct programmer involvement. Automatic memory management reduces the likelihood of memory leaks, dangling

pointers, and other memory-related programming errors. While garbage collection introduces a small amount of runtime overhead, it greatly improves application stability and reduces the possibility of accidental memory corruption.

C++ relies primarily on manual memory management, although modern C++ provides smart pointers and resource management techniques that simplify this process. Because programmers have direct access to memory, C++ applications can achieve excellent performance when written correctly. However, improper memory management can introduce vulnerabilities such as memory leaks, dangling pointers, buffer overflows, and undefined behavior. These vulnerabilities have historically been responsible for many software security issues. As a result, developing secure C++ applications often requires greater experience and careful attention to memory allocation and resource management.

String handling also demonstrates an important difference between the two languages. Java uses the immutable *String* class. Once a string has been created, its contents cannot be modified. This design helps prevent many common programming errors and memory corruption vulnerabilities because string objects cannot accidentally overwrite existing memory. However, immutable strings may remain in memory until garbage collection occurs, making them less suitable for storing highly sensitive information such as passwords. For that reason, Java developers frequently use *char[]* when handling confidential data because the contents of the array can be cleared immediately after use.

C++ provides several methods for working with strings, including traditional character arrays and the modern *std::string* class. Character arrays offer flexibility but are susceptible to

buffer overflow vulnerabilities if programmers fail to validate input correctly. Modern *std::string* objects provide much safer memory management than traditional C-style strings and greatly reduce the likelihood of overflow errors. Even so, C++ still provides programmers with greater direct access to memory than Java, making secure coding practices especially important.

Exception handling is another important distinction between the two implementations. Java requires programmers to properly handle many exceptions before code can compile successfully. In this project, *InterruptedException* must be handled whenever the program pauses execution using *Thread.sleep()*. This requirement encourages developers to anticipate runtime problems and implement appropriate recovery mechanisms. Although mandatory exception handling may require additional code, it generally produces more reliable and maintainable applications.

C++ also supports exception handling through *try* and *catch* blocks, but many exceptions are optional rather than enforced by the compiler. This flexibility allows developers to write highly optimized code but also increases the possibility that certain errors will not be handled appropriately. The responsibility for identifying and managing exceptional conditions falls primarily on the programmer rather than the language itself.

From a security perspective, Java is generally considered the less vulnerable language for most application development. Automatic memory management, strong type checking, array bounds checking, mandatory exception handling, and execution within the Java Virtual Machine collectively reduce many common software vulnerabilities. These built-in protections make Java

an excellent choice for enterprise software, financial systems, web applications, and other environments where reliability and security are major priorities.

C++, however, offers capabilities that Java cannot always match. Direct hardware access, deterministic performance, and fine-grained memory control make C++ the preferred choice for operating systems, embedded devices, graphics engines, scientific computing, and other performance-critical applications. When experienced developers follow secure coding practices and carefully manage memory, C++ applications can be extremely reliable. Nevertheless, the language provides fewer built-in safeguards than Java, making programmer expertise much more important.

After comparing both implementations, Java would generally be considered the less vulnerable option for this particular project. The Java implementation provides automatic memory management, built-in runtime safety checks, and stronger protection against common memory-related vulnerabilities. While the C++ implementation offers greater execution speed and lower resource usage, it also introduces additional risks if memory and thread management are not performed correctly. Because this project focuses on demonstrating concurrency concepts rather than maximizing performance, Java provides a simpler and safer implementation while still satisfying all of the project requirements.

Overall, both implementations successfully demonstrate the use of concurrency through synchronized threads. The Java version emphasizes portability, safety, and ease of development, while the C++ version emphasizes efficiency, flexibility, and programmer control. Each language has strengths that make it appropriate for different types of software. Choosing between Java and

C++ ultimately depends on the goals of the application, the importance of execution speed, available system resources, security requirements, and the experience of the development team. For this assignment, both programs effectively demonstrate the fundamental concepts of concurrency while highlighting the unique advantages and trade-offs of each programming language.

Conclusion

This portfolio project provided an opportunity to demonstrate the use of concurrency in both Java and C++ while comparing how each language implements multithreaded programming. Although both applications completed the same task by creating two synchronized threads, the implementation details highlighted important differences in performance, memory management, security, and ease of development.

The Java implementation emphasized safety, portability, and simplified thread management through the Java Virtual Machine and automatic memory management. These features reduce many common programming errors and make Java an excellent choice for enterprise and business applications where reliability is essential. The C++ implementation offered greater control over system resources and higher execution performance, but it also required more careful management of memory and thread synchronization.

Completing both implementations reinforced the importance of proper thread management, synchronization using the *join()* method, and selecting the appropriate language based on project requirements. Overall, this project demonstrated that both Java and C++ are capable of implementing

concurrent applications successfully, but each language offers unique advantages depending on the performance, security, and maintenance needs of the software being developed.

References

Oracle. (2024). *Java Platform, Standard Edition documentation*. https://docs.oracle.com/en/java/

ISO. (2020). *Programming languages — C++ (ISO/IEC 14882:2020)*. International Organization for Standardization.